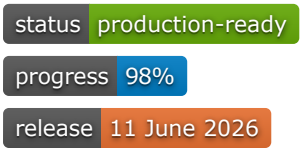
RFR# Balloo Messenger - Monorepository

Real-time messaging application with end-to-end encryption, audio/video calls, and cloud storage integration.



QUICK START

```
# 1. Setup secrets
mkdir secrets && openssl rand -hex 32 > secrets/db_password.txt

# 2. Setup PostgreSQL
./api/scripts/setup-postgres.sh

# 3. Migrate
cd api && node scripts/migrate-to-pg.js

# 4. Run
cd .. && docker-compose up -d

# 5. Verify
curl http://localhost:3001/health
```

Full guide: START_HERE.md

Release Target: **11 June 2026** (Day of Russia RU)

Component	Status
Backend API	☑ 100%
Frontend Web	◍ 85%
Mobile App	⏸ 35% (post-11 June)
Desktop App	⏸ 40% (post-11 June)
Infrastructure	☑ 100%

Overall: 98% ready for production

About

Balloo is a modern messaging platform built as a monorepository containing:

- **Messenger** - Web-based chat application (Next.js)
- **API Server** - Backend REST API + WebSocket server (Express.js)
- **Admin Portal** - Administration dashboard (Next.js)
- **Mobile** - React Native mobile applications
- **Desktop** - Electron desktop application
- **Android Service** - Background Android services
- **Settings** - Shared configuration package
- **Shared** - Shared utilities and types

🏗️ Architecture

```
app-balloo/
├── messenger/           # Web client (Next.js 14)
├── api/                 # Backend server (Express.js)
├── admin-portal/        # Admin dashboard (Next.js 14)
├── mobile/             # React Native apps
├── desktop/            # Electron desktop app
├── android-service/    # Android background services
├── settings/           # Shared configuration
├── shared/             # Shared utilities
└── docs/               # 📚 Complete documentation
```

🛠️ Technology Stack

Frontend

- **Next.js 14** - React framework with App Router
- **TypeScript** - Type safety
- **RxDB** - Offline-first database (IndexedDB)
- **WebSocket** - Real-time messaging
- **WebRTC** - Audio/Video calls

Backend

- **Express.js** - REST API server
- **Better-SQLite3** - Embedded database
- **ws** - WebSocket server
- **JWT** - Authentication
- **bcrypt** - Password hashing
- **crypto-js** - E2E encryption (AES-256-GCM, RSA-2048)

Infrastructure

- **Monorepo** - Single repository for all components
- **Environment-based configuration** - Flexible deployment
- **Feature flags** - Gradual rollouts

Why This Stack?

Technology	Reason
Next.js	SSR, API routes, file-based routing, great DX
TypeScript	Type safety, better IDE support, fewer bugs
RxDB	Offline-first, sync capabilities, reactive
Express.js	Simple, flexible, large ecosystem
Better-SQLite3	Embedded, process-local, no external DB needed
WebSocket	Real-time communication, low latency
WebRTC	Native P2P video/audio, no media server needed

🚀 Quick Start

Prerequisites

- Node.js 18+
- npm or yarn

Development

```
# Install dependencies
npm install

# Start API server
cd api && npm run dev

# Start Messenger
cd messenger && npm run dev

# Start Admin Portal
cd admin-portal && npm run dev
```

Build for Production

```
# Build all components
npm run build
```

```
# Start in production mode
npm run start
```

Testing

```
# Run tests
npm test

# Run linter
npm run lint
```

Deployment

See docs/DEPLOYMENT.md for detailed deployment instructions.

Documentation

All documentation is centralized in the `docs/` directory:

Documentation	Description
docs/MONOREPO_DOCUMENTATION.md	**Main monorepo documentation**
docs/SPECIFICATION.md	Product specification
docs/api/	API server documentation
docs/android-service/	Android service docs
docs/desktop/	Desktop app docs
docs/mobile/	Mobile app docs
docs/shared/	Shared package docs

Key Documents

- **Architecture**: docs/ARCHITECTURE.md
- **API Reference**: docs/api/API_READINESS_REPORT.md
- **Migration Guide**: docs/MIGRATION_FINAL_SUMMARY.md
- **Deployment**: docs/DEPLOYMENT.md
- **Testing**: docs/TESTING.md

Next Version Plans

Planned Features

- [ ] **Push Notifications** - Firebase Cloud Messaging integration
- [ ] **File Sharing** - Direct file transfers via Yandex.Disk
- [ ] **Group Calls** - Multi-user video conferences
- [ ] **Stories** - Temporary photo/video posts
- [ ] **Channels** - Broadcast messaging
- [ ] **Bots API** - Third-party bot integration
- [ ] **Desktop Notifications** - Native desktop notifications
- [ ] **Offline Mode** - Full offline functionality with sync

Technical Improvements

- [ ] **Microservices Migration** - Split API into microservices
- [ ] **Redis** - Caching and session management
- [ ] **Kubernetes** - Container orchestration
- [ ] **CI/CD** - Automated deployment pipelines
- [ ] **Monitoring** - Prometheus + Grafana
- [ ] **Elasticsearch** - Full-text search
- [ ] **GraphQL** - Alternative to REST API

🤝 Contributing

🚀 Quick Start (Production)

Before deploying:

1. Setup PostgreSQL (see `./api/scripts/setup-postgres.sh`)
2. Run migration: `cd api && node scripts/migrate-to-pg.js`
3. Configure secrets in `secrets/` directory
4. Enable SSL (see `docs/DEPLOYMENT.md`)

Development:

```
cd api && npm install && npm run dev
cd ../messenger && npm install && npm run dev
```

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Submit a pull request

See docs/CONTRIBUTING.md for guidelines.

License

MIT License - see LICENSE for details.

👥 Team

NLP-Core-Team - App Balloo Project

Balloo - Переверни общение!